

De React a SolidTV

Una home de streaming, paso a paso

Leidy Yohana Rodriguez Fuentes

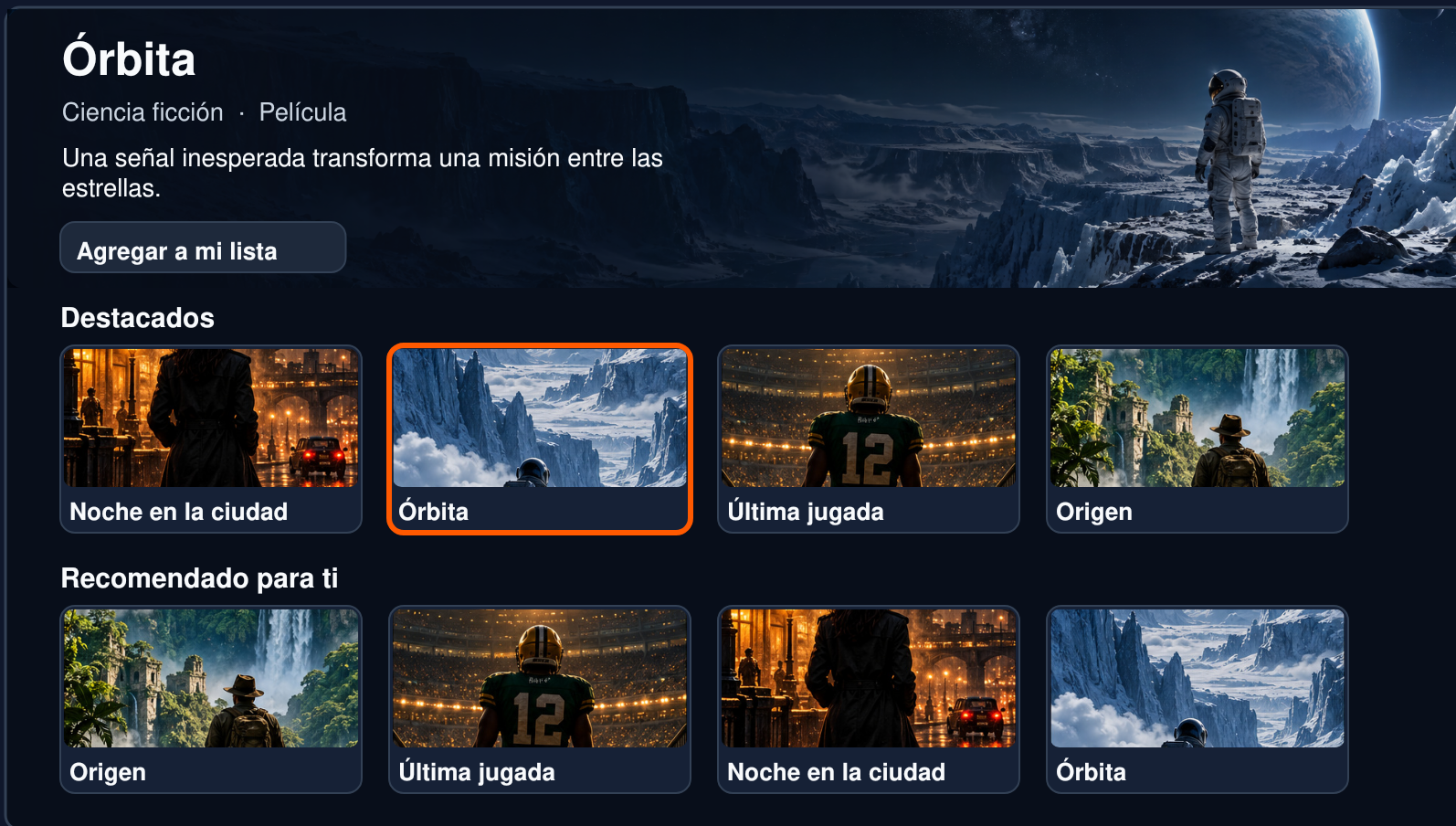
Contenido de la sesión

Una misma pantalla nos acompaña desde la primera card hasta la home.

- | | | |
|----|---------------------------|---|
| 01 | La experiencia | Hero, dos carruseles y favoritos |
| 02 | Los conceptos | Reactividad, WebGL y nodos de SolidTV |
| 03 | Los componentes | Card, carrusel, foco y estado |
| 04 | La página completa | Conectar datos, navegación y presentación |

La home que vamos a construir

Dos carruseles. Un hero que responde al foco. Favoritos compartidos.



01 NAVEGAR

Recorrer dos carruseles

02 ENFOCAR

Resaltar la card activa

03 ACTUALIZAR

Cambiar imagen e info del hero

04 GUARDAR

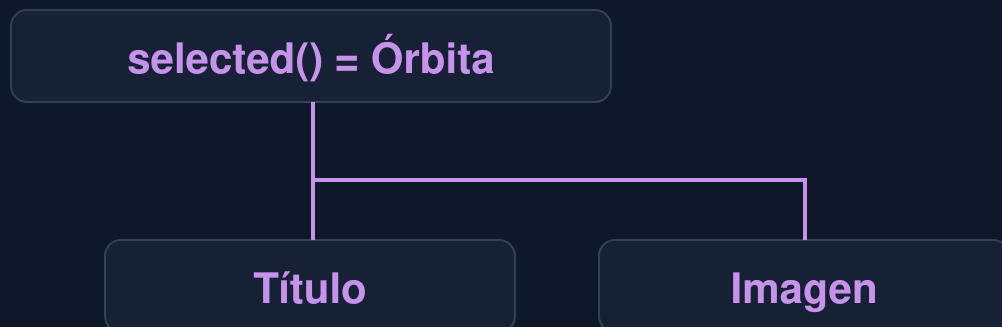
Agregar y quitar favoritos

Contenido bajo demanda: carruseles, foco, estilos y reactividad en una sola pantalla.

Reactividad de grano fino y WebGL

Al enfocar Órbita, cambian los datos del hero y su representación en pantalla.

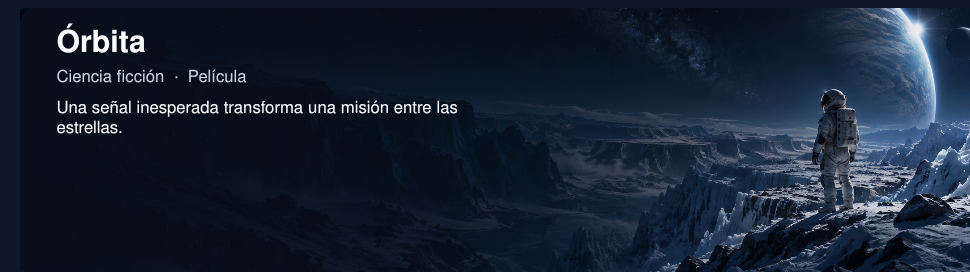
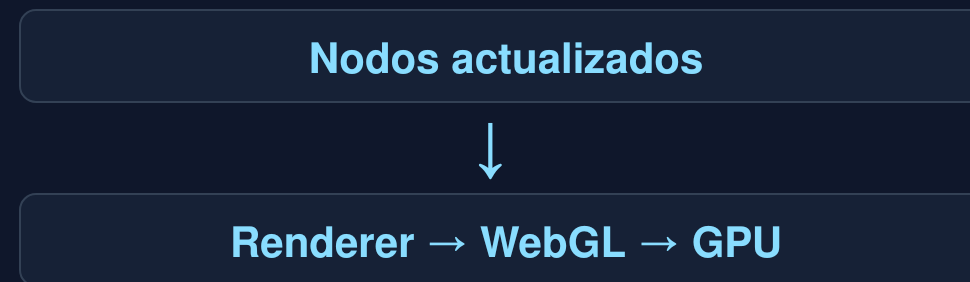
SOLIDJS · DEPENDENCIAS



Las lecturas conectan el estado con sus consumidores.

El catálogo no depende de `selected()`.

RENDERER WEBGL · DIBUJO



La GPU dibuja imágenes, textos y formas.

Reactividad: qué valores se actualizan. WebGL: cómo se dibuja la escena.

React y Solid actualizan la interfaz

El mismo resultado visible: el título del hero pasa de Noche a Órbita.

React

Estado cambia

setSelected(orbita)



Render y reconciliación

React calcula y compara la salida



Commit

Aplica los cambios necesarios

Solid

Signal cambia

setSelected(orbita)



Consumidores registrados

Se reevalúan los bindings que lo leen



Propiedades actualizadas

Título, imagen y descripción del hero

Cambiar un dato no significa reconstruir toda la interfaz.

Quién hace qué en SolidTV

Cada capa resuelve una parte de la misma experiencia.

SolidJS

Dependencias reactivas

Signals, memos y effects

SolidTV

Interfaz declarativa

JSX, nodos y foco

Renderer

Dibujo de la escena

Texturas, texto y GPU

`solid-js`



`@solidtv/solid`



`@solidtv/renderer`

Control remoto → estado → consumidores → nodos → pantalla

HTML y los nodos de SolidTV

La intención se conserva. La escena WebGL usa nodos del render.

En HTML	En esta UI con SolidTV
div / section	view para contener y agrupar
p / span / h1	text con fontSize, color y posición
img	view con src, width y height
button	Componente propio: view + text + onEnter
className / CSS	style y propiedades del nodo

INCORRECTO EN LA ESCENA WEBGL

```
<button className="favorite">
  Agregar a mi lista
</button>
```

CORRECTO PARA NUESTRO BOTÓN

```
<view style={buttonStyle}
  onEnter={handleEnter}>
  <text>Agregar a mi lista</text>
</view>
```

Derivar, memorizar o ejecutar un efecto

`selected()` lee el contenido enfocado. A partir de ese dato podemos:

Función derivada

Formatear el género para el hero.

```
const genreLabel = () =>
  selected().genre + " · Película";
```

`createMemo`

Alternativa con caché entre cambios.

```
const genreLabel = createMemo(() =>
  selected().genre + " · Película"
);
```

`createEffect`

Sincronizar con algo externo: la consola.

```
createEffect(() => {
  console.log("Foco:", selected().id);
});
```

PosteCard: imagen y título

Primero construimos la pieza que se repite en ambos carruseles.

POSTERCARD.JSX

```
function PosteCard(props) {  
  return <view style={cardStyle}>  
    <view x={3} y={3} width={258}  
      height={121} borderRadius={5}  
      src={props.movie.poster} />  
    <text x={8} y={132} fontSize={22}>  
      {props.movie.title}  
    </text>  
  </view>;  
}
```



props.movie

id · title · poster
hero · genre · description

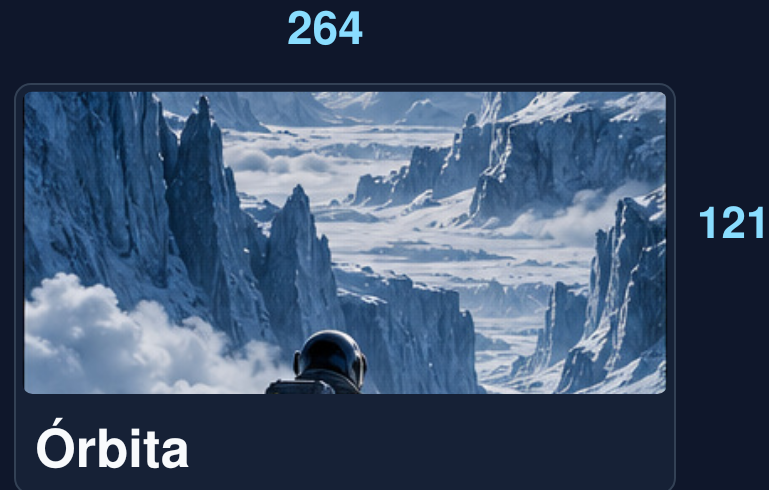
<view> crea un nodo; <text> muestra texto. La card recibe datos por props.

Dimensiones y estilos de la card

La imagen termina en $y = 124$; el título empieza en $y = 132$.

ESTILO BASE REUTILIZABLE

```
const cardStyle = {  
  width: 264, height: 164,  
  color: 0x162136ff,  
  borderRadius: 8,  
  border: {  
    width: 3, color: 0x334155ff,  
  },  
};
```



style define la base. Las props JSX pueden conectar valores reactivos.

Un carrusel con For y Show

For repite la card. Show decide si hay contenido que mostrar.

DENTRO DE CAROUSEL

```
<Show when={props.movies.length > 0}>
  <view display="flex" gap={24}>
    <For each={props.movies}>
      {movie => <PosterCard movie={movie} />}
    </For>
  </view>
</Show>
```

For

Una card por contenido.

Show

Oculto la lista si está vacía.



Noche en la ciudad



Órbita



Última jugada



Origen

El foco recorre y resalta las cards

Row navega; el color del wrapper resalta la imagen y se restaura al perder foco.

WRAPPER DE IMAGEN · EN POSTERCARD

```
const imageWrapper = {  
  color: 0x00000000,  
  borderRadius: 8,  
  $focus: { color: 0xff5900ff },  
};
```

DENTRO DE POSTERCARD · WRAPPER DE FOCO

```
<view style={cardStyle}>  
  <view width={264} height={127}  
    {...imageWrapper}>  
    <view x={3} y={3} width={258}  
      height={121} borderRadius={5}  
      src={props.movie.poster}  
      color={0xffffffff} skipFocus />  
    </view>  
    <text x={8} y={132} fontSize={22}>  
      {props.movie.title}</text>  
    </view>
```



Noche en la ciudad



Órbita



Última jugada



Origen

Imagen dentro, a 3 px.
color: 0xffffffff; skipFocus

Aplica {...imageWrapper} al contenedor sin src. Al perder foco, vuelve a transparente.

Un signal para el contenido seleccionado

Home pasa onSelect a Carousel; Carousel lo pasa a cada PosterCard.

EN HOME · createSignal DESDE solid-js

```
const [selected, setSelected] =  
  createSignal(movies[0]);  
  
<Carousel movies={movies}  
  onSelect={setSelected} />
```

CAROUSEL CONECTA POSTERCARD (DIAP. 9)

```
<PosterCard movie={movie}  
  onSelect={props.onSelect} />  
// Wrapper de imagen de PosterCard:  
onFocusChanged={focused => {  
  if (focused) props.onSelect(props.movie);  
}}
```



Órbita

selected() = Órbita

true: actualiza selected()
false: conserva selección

El hero responde a la selección

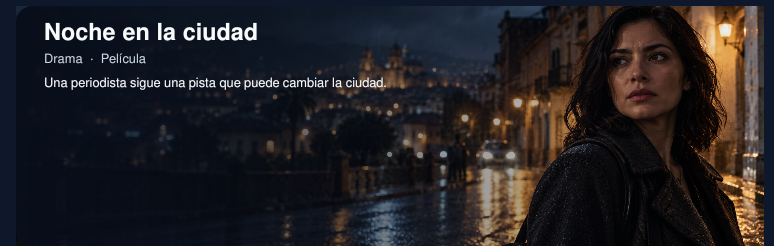
La imagen ocupa todo el ancho. El gradiente permite leer el texto encima.

EN HOME

```
<Hero movie={selected()} />
```

DENTRO DE HERO · FRAGMENTO

```
<view width={1280} height={244}>  
  <view width={1280} height={244}  
    src={props.movie.hero} />  
  <view width={1280} height={244}  
    color={0x080e1aff}  
    colorLeft={0x080e1aff} colorRight={0x080e1a00} />  
  <text x={48} y={18} fontSize={40}>  
    {props.movie.title}  
  </text>  
  <text x={48} y={116} width={570} contain="width"  
    fontSize={22}>{props.movie.description}</text>  
</view>
```



↓ **Nuevo foco**

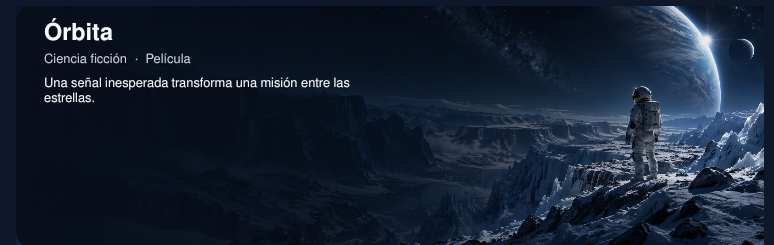


Imagen + gradiente + texto
Lee props.movie dentro del JSX.

Un botón para agregar o quitar favoritos

Subir al botón conserva la selección. Enter cambia su estado de favorito.

DENTRO DE FAVORITEBUTTON

```
<view style={buttonStyle}
  onEnter={() => {
    props.onToggle();
    return true;
  }}>
  <text x={14} y={12} fontSize={22}>
    {props.favorite
      ? "Guardado" : "Agregar a mi lista"}
  </text>
</view>
```

Agregar a mi lista

Subir al botón

Agregar a mi lista

Enter

Guardado

El botón conserva el foco. Pulsar Enter otra vez quita el favorito.

Favoritos compartidos con createStore

La identidad es movie.id, aunque la película aparezca en dos carruseles.

EN HOME

```
import { createStore } from "solid-js/store";
const [favorites, setFavorites] =
  createStore({});
const toggleFavorite = id =>
  setFavorites(id, value => !value);
```



Órbita

CONECTAR LA SELECCIÓN CON EL BOTÓN

```
<FavoriteButton
  favorite={favorites[selected().id]}
  onToggle={() => toggleFavorite(selected().id)}
/>
```

favorites.orbita = true

También se pasa a cada card:
favorite={favorites[movie.id]}

Navigation: botón y dos carruseles

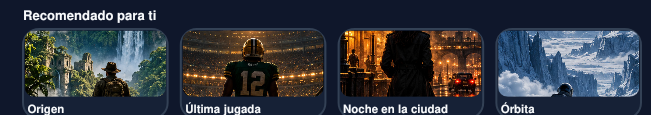
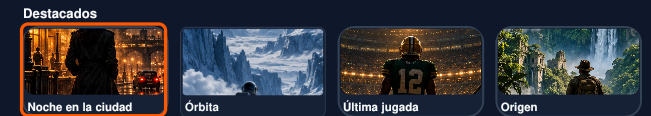
Componente propio del ejemplo: agrupa los elementos que reciben foco.

NAVIGATION.JSX

```
function Navigation(props) {  
  return <Column x={48} y={188} gap={24}  
    autofocus selected={1} scroll="none">  
    <FavoriteButton  
      favorite={props.favorites[props.selected.id]}  
      onToggle={() => props.onToggle(props.selected.id)} />  
    <Carousel title="Destacados" movies={movies}  
      favorites={props.favorites}  
      onSelect={props.onSelect} />  
    <Carousel title="Recomendado para ti"  
      movies={recommended} favorites={props.favorites}  
      onSelect={props.onSelect} />  
  </Column>;  
}
```

Nota: también puedes guardar las secciones en un array y usar `<For>` para crear cada `Carousel`.

Agregar a mi lista



`selected={1}`: inicia en el primer carrusel.

Home conecta los componentes

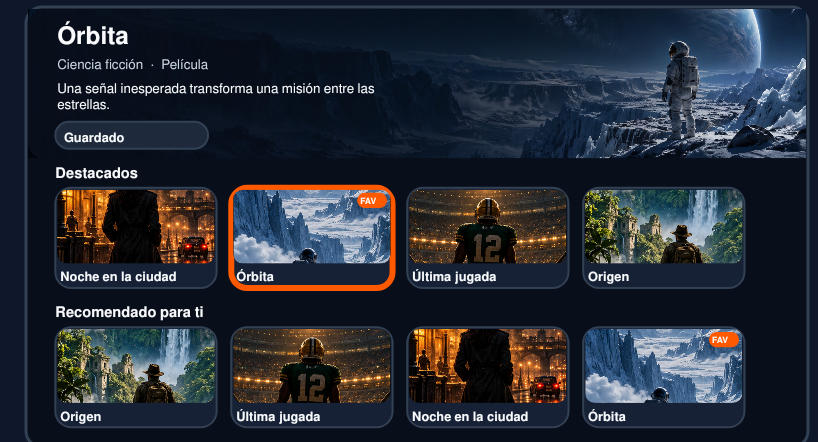
Un estado de selección y un estado de favoritos para toda la página.

ESTADO EN EL CUERPO DE HOME

```
useFocusManager();
const [selected, setSelected] =
  createSignal(movies[0]);
const [favorites, setFavorites] =
  createStore({});
const toggleFavorite = id =>
  setFavorites(id, value => !value);
```

HERO + NAVIGATION (DEFINIDO EN LA DIAPOSITIVA ANTERIOR)

```
<Hero movie={selected()} />
<Navigation selected={selected()}
  favorites={favorites}
  onSelect={setSelected}
  onToggle={toggleFavorite} />
```



Nuestro Navigation:

FavoriteButton

Carousel: Destacados

Carousel: Recomendado

Lo que construimos y lo que aprendimos

Una home pequeña reúne las bases que después escalan a una app de TV.

Card

Nodos, props y estilos

Carrusel

For, Show y navegación con Row

Hero

Signals y lecturas reactivas

Favoritos

Estado por ID con createStore

Home

Composición y navegación con Column

¡Gracias!

¿Preguntas?

Ahora el foco lo tienen ustedes.

